New Horizons
Centro de Capacitación

Práctica: Validación con FluentValidation y ProblemDetails

Paso a paso — Blindar la API con validadores automáticos, middleware de excepciones y certificar con pruebas

Curso	Técnico de Desarrollo Full Stack con C# y Angular
Sesión	Sesión 7 — Validación, Middleware e Introducción a Angular
Docente	Edwin Barrientos
Duración estimada	60–90 minutos
Nivel	Básico (guía principal de la sesión)
Entorno	.NET 8 SDK · Visual Studio 2022 Community o VS Code · PostgreSQL o SQL Server Express

1. Objetivos

- Instalar y configurar el paquete FluentValidation.AspNetCore en la solución.
- Crear validadores fuertemente tipados con reglas de negocio claras para CrearSolicitudDto.
- Configurar el filtro de validación para emitir respuestas de error estructuradas.
- Implementar un middleware global de manejo de excepciones devolviendo el estándar RFC 7807 ProblemDetails.
- Automatizar las pruebas de certificación de endpoints con un archivo .http completo.

2. Desarrollo paso a paso

Paso 1 — Instalar FluentValidation

Instalamos los paquetes en Application y en Api.

terminal

```
dotnet add src/SolicitudesApp.Application package FluentValidation.DependencyInjectionExtensions
dotnet add src/SolicitudesApp.Api package FluentValidation.AspNetCore
```

Paso 2 — Crear CrearSolicitudValidator

En Application/Validaciones/CrearSolicitudValidator.cs definimos las reglas de validación de entrada.

CrearSolicitudValidator.cs

```
using FluentValidation;
using SolicitudesApp.Application.Dtos;

namespace SolicitudesApp.Application.Validaciones;

public class CrearSolicitudValidator : AbstractValidator<CrearSolicitudDto>
{
    public CrearSolicitudValidator()
    {
        RuleFor(x => x.Motivo)
            .NotEmpty().WithMessage("El motivo de la solicitud es obligatorio.")
            .MaximumLength(200).WithMessage("El motivo no puede exceder los 200 caracteres.");

        RuleFor(x => x.CentroCostoId)
            .GreaterThan(0).WithMessage("Debe seleccionar un centro de costo válido.");

        RuleFor(x => x.Detalles)
            .NotEmpty().WithMessage("La solicitud debe incluir al menos un detalle de gasto.");

        RuleForEach(x => x.Detalles).ChildRules(detalle =>
        {
            detalle.RuleFor(d => d.Concepto)
                .NotEmpty().WithMessage("El concepto del gasto es obligatorio.");

            detalle.RuleFor(d => d.Importe)
                .GreaterThan(0).WithMessage("El importe debe ser mayor a cero.");
        });
    }
}
```

Paso 3 — Middleware global de excepciones con ProblemDetails

En Api/Middlewares/ExceptionHandlingMiddleware.cs capturamos cualquier error no controlado y respondemos con RFC 7807.

ExceptionHandlingMiddleware.cs

```
using System.Net;
using System.Text.Json;
using Microsoft.AspNetCore.Mvc;
using SolicitudesApp.Domain.Excepciones;

namespace SolicitudesApp.Api.Middlewares;

public class ExceptionHandlingMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<ExceptionHandlingMiddleware> _logger;

    public ExceptionHandlingMiddleware(RequestDelegate next, ILogger<ExceptionHandlingMiddleware>
logger)
    {
        _next = next;
        _logger = logger;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        try
        {
            await _next(context);
        }
        catch (ReglaDominioException ex)
        {
            _logger.LogWarning("Regla de dominio violada: {Mensaje}", ex.Message);
            await EscribirRespuestaAsync(context, HttpStatusCode.BadRequest, "Violación de regla
de negocio", ex.Message);
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Error inesperado en el servidor");
            await EscribirRespuestaAsync(context, HttpStatusCode.InternalServerError, "Error
interno del servidor", "Ha ocurrido un error inesperado. Contacte a soporte.");
        }
    }

    private static Task EscribirRespuestaAsync(HttpContext context, HttpStatusCode codigo, string
titulo, string detalle)
    {
        context.Response.ContentType = "application/problem+json";
        context.Response.StatusCode = (int)codigo;

        var problema = new ProblemDetails
        {
            Status = (int)codigo,
            Title = titulo,
            Detail = detalle,
            Instance = context.Request.Path
        };

        return context.Response.WriteAsync(JsonSerializer.Serialize(problema));
    }
}
```

Paso 4 — Registrar validadores y middleware en Program.cs

Activamos FluentValidation y colocamos el middleware en la primera posición del pipeline.

Program.cs

```
builder.Services.AddValidatorsFromAssemblyContaining<CrearSolicitudValidator>();

var app = builder.Build();

app.UseMiddleware<ExceptionHandlingMiddleware>(); // Primero en la cadena
app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

Paso 5 — Certificar el Backend con solicitudes-certificacion.http

Ejecutamos pruebas de campos vacíos, importes negativos y violaciones de estado confirmando que devuelven 400 ProblemDetails.

solicitudes-certificacion.http

```
### Probar validación fallida: motivo vacío e importe cero
POST https://localhost:5001/api/solicitudes
Content-Type: application/json

{
  "motivo": "",
  "centroCostoId": 0,
  "detalles": [
    { "concepto": "", "importe": -50.0 }
  ]
}
### Debe retornar HTTP 400 con los errores estructurados
```

3. Retos adicionales

1. Agrega una regla personalizada que valide que el importe total de la solicitud no supere los S/ 50,000.
2. Añade la propiedad TraceId al objeto ProblemDetails para correlacionar logs de producción.
3. Configura Swagger para documentar que los endpoints retornan 400 ProblemDetails.
4. Escribe una prueba unitaria que verifique que CrearSolicitudValidator falla cuando la lista de detalles está vacía.

4. Entregables

- FluentValidation rechazando datos corruptos con mensajes en español y campos identificados.
- Middleware global capturando errores no controlados y devolviendo JSON bajo RFC 7807.
- Archivo de certificación .http con pruebas de validación ejecutadas con éxito.
- Backend certificado al 100% listo para ser consumido por el frontend.